CS501 Final Project Report: Micro Chat

Fan Lu, Yuhang Xu, Yuchen Yang

December 14, 2025

Abstract

This report documents the engineering and development of "Micro Chat," a native Android application aimed at reducing linguistic barriers in professional communication. This project implements a robust MVVM architecture using Jetpack Compose for the UI and Firebase for real-time data synchronization. A key technical feature is the integration of Generative AI via a secure proxy backend to provide real-time translation and semantic refinement.

Contents

1 Technical Implementation **2**
- 1.1 Architecture Design . . . 2
- 1.2 API Integration & Backend Strategy . . . 2
  - 1.2.1 Firebase Ecosystem . . . 2
  - 1.2.2 Secure AI Proxy Implementation . . . 2
- 1.3 Native Features . . . 3
- 1.4 Jetpack Compose Implementation . . . 3

2 Problem Definition and Solution **3**
- 2.1 Problem Statement . . . 3
- 2.2 Engineering Challenges & Solutions . . . 3

3 Team Process and Methodology **5**
- 3.1 Collaboration and Tooling . . . 5
- 3.2 DevOps and Automation . . . 5
- 3.3 Development Lifecycle . . . 5
- 3.4 QA Strategy . . . 6

4 Role of AI in Development **6**
- 4.1 Usage Scenarios . . . 6
- 4.2 Critical Reflection . . . 6
  - 4.2.1 Benefits . . . 6
  - 4.2.2 limitations and Human Oversight . . . 6

1 Technical Implementation

Our goal was to build a scalable, maintainable application that adheres to current industry standards. We prioritized the separation of concerns and reactive data flow.

1.1 Architecture Design

We adopted the Model-View-ViewModel (MVVM) pattern, structured around Clean Architecture principles. This decision was driven by the need to decouple our business logic from the UI, facilitating easier unit testing and preventing "God Activities."

- **Presentation Layer (UI):** We utilized **Jetpack Compose** exclusively, moving away from legacy XML layouts. This allowed us to build a declarative UI where components react seamlessly to state changes exposed by ViewModels via **StateFlow**.
- **Domain Layer:** We implemented **Repositories** (e.g., `ChatRepository`, `TranslationRepository`) to mediate data operations. These repositories serve as the "single source of truth," abstracting whether data comes from the local cache or the remote Firestore database.
- **Data Layer:** This layer handles interactions with **Firebase** and our custom proxy server.
- **Dependency Injection (DI):** We used **Hilt** to manage dependency injection. This significantly reduced boilerplate code for initializing ViewModels and Repositories and made swapping data sources for testing much simpler.

1.2 API Integration & Backend Strategy

To support the core functionality, we integrated several external services:

1.2.1 Firebase Ecosystem

We leveraged Firebase for its serverless capabilities, allowing us to focus on client-side logic:

- **Authentication:** We implemented OAuth flows (Google Sign-In) and standard Email/-Password authentication.
- **Cloud Firestore:** We chose this NoSQL database for its listener capabilities. By attaching snapshot listeners to the `messages` sub-collections, we achieved real-time chat updates without needing to implement long-polling or WebSockets manually.
- **Storage:** Used for hosting large binary assets like user avatars and audio files.

1.2.2 Secure AI Proxy Implementation

A major architectural decision was to **not** call OpenAI APIs directly from the Android client to prevent API key leakage (a common vulnerability in mobile apps). Instead, we deployed a lightweight Java middleware service on Google Cloud Run.

- **Request Handling:** The Android app sends a payload to our server.
- **Prompt Engineering:** The server injects system-level instructions (e.g., "You are a helpful translator...") before forwarding the request to OpenAI's Chat Completions API.
- **Transcription:** Voice notes are processed via the Whisper model (`gpt-4o-mini-transcribe`) on the backend, returning raw text to the client.

1.3 Native Features

We utilized Android's native `TextToSpeech` engine rather than a cloud API to reduce latency and data usage. The engine is instantiated within the `ViewModel` scope to manage lifecycle correctly.

1.4 Jetpack Compose Implementation

Moving to 100% Compose allowed for rapid UI iteration.

- **Responsiveness:** We utilized `BoxWithConstraints` to dynamically adjust layout logic based on available screen real estate (differentiating between phone and tablet breakpoints).
- **State Hoisting:** We strictly followed state hoisting patterns, ensuring that Composables remain stateless functions that simply render data provided by the `ChatDetailViewModel`.
- **Theming:** A central `Theme.kt` file manages our Material 3 design system, handling dark/light mode switching automatically.

2 Problem Definition and Solution

2.1 Problem Statement

In professional environments, non-native speakers often struggle with the nuance and tone required for business communication. Standard translation tools often yield literal translations that lack context, leading to "Imposter Syndrome" or miscommunication. Our objective was to create a tool that not only translates text but enhances the professional quality of the communication.

2.2 Engineering Challenges & Solutions

- **Secure AI Proxying:** Client calls the Java proxy for translation/summarization/TTS instead of hitting OpenAI directly. *Solution:* Keep API keys server-side; validate payloads and return minimal JSON to the app.
- **Inline Translation Latency:** Per-message translations flow through the proxy and update UI state in Compose. *Solution:* Cache by message key in the ViewModel, stream loading states, and fall back to originals when requests fail.

- **Auto-Translate Control:** User preference is stored in DataStore and applied per incoming message. *Solution:* Honor the toggle, skip sender's own messages, and allow per-message suppression when the user hides a translation.
- **Voice Transcribe-then-Translate Robustness:** Voice notes are uploaded, transcribed, then translated automatically. *Solution:* Mark UI with loading/error states, cap retries, and keep the original transcript to retry translation without re-uploading audio.
- **Summary Faithfulness:** Thread summaries run through the same proxy and return concise text. *Solution:* Format messages with sender names before sending, surface errors to the UI, and clear stale summaries on demand.
- **Resilience & Transparency:** Network failures are common on mobile. *Solution:* Emit user-friendly toasts/messages for translate/transcribe/summarize errors, log events for debugging, and preserve originals so users can continue chatting even when AI calls fail.

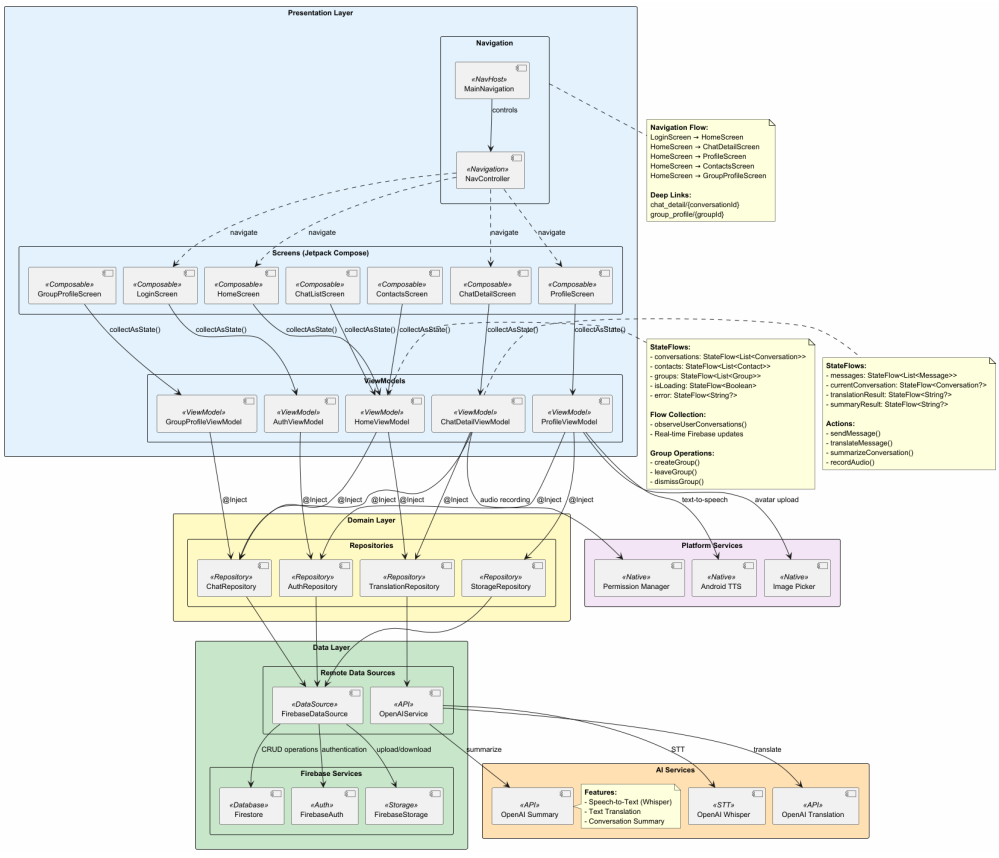

Figure 1: Diagram of Technical Architecture

Retrospective

The most significant lesson was the importance of **architecture over features**. By spending time setting up Hilt and Clean Architecture early, we avoided spaghetti code when adding complex features like AI integration later in the semester.

3 Team Process and Methodology

We utilized an Agile-inspired development process, prioritizing iterative delivery and continuous feedback. Our workflow was organized into one-week sprints, allowing for regular assessment of progress and distinct separation of concerns.

3.1 Collaboration and Tooling

- **Version Control Strategy:** We utilized a Feature Branch workflow. Developers created dedicated branches for specific tickets, which required a Pull Request (PR) and code review from another member before merging into the `main` branch.

3.2 DevOps and Automation

We established a Continuous Integration/Continuous Deployment (CI/CD) pipeline to streamline our backend release process:

- **GitHub Actions:** We configured a workflow that triggers automatically upon any push to the `yyc/OpenAI` branch. This workflow runs our build scripts and linting checks to ensure code quality.
- **Automated Deployment:** Upon a successful build, the pipeline automatically deploys the latest image to Google Cloud. This ensured that our live environment was always synchronized with our codebase, allowing for immediate testing of new features by the whole team.

3.3 Development Lifecycle

- **Phase 1: MVP.** We focused entirely on the "Happy Path"—User Auth, Chat Listing, and Basic Messaging. This validated our database schema and established the core architecture early to avoid technical debt.
- **Phase 2: Integration.** Once the chat infrastructure was stable, we integrated external services, specifically the OpenAI proxy and the "Polish" message feature. This phase relied heavily on pair programming to solve async connectivity issues.
- **Phase 3: Refinement.** The final sprint was dedicated to "hardening" the application. We utilized this time for edge-case handling (e.g., managing state during no internet connection), UI standardization, and code cleanup based on previous sprint retrospectives.

3.4 QA Strategy

- **Unit Testing:** We wrote JUnit tests for our backend JSON parsers to ensure the AI response format matched our Android data models.
- **UI Testing:** We use UI tests to verify the authentication flow works as expected.

4 Role of AI in Development

We treated Generative AI as a "force multiplier" rather than a replacement for engineering. It significantly accelerated our velocity but required careful auditing.

4.1 Usage Scenarios

- **Prototyping:** We used ChatGPT to generate the initial scaffolding for composables. For example: *"Create a Material 3 LazyColumn for a chat interface with distinct styles for sent and received messages."*
- **Infrastructure:** AI was excellent at generating standard boilerplate, such as Retrofit service interfaces and Firebase data classes.
- **Localization:** We automated the translation of our `strings.xml` file, allowing us to launch with support for 5 languages immediately.

4.2 Critical Reflection

4.2.1 Benefits

The primary benefit was velocity. Tasks that usually take hours of documentation reading (like setting up the exact syntax for a Firestore complex query) were reduced to minutes. It acted effectively as an on-demand StackOverflow.

4.2.2 limitations and Human Oversight

However, reliance on AI introduced risks that required senior-level intervention:

- **Context Awareness:** AI often suggested architectural patterns that contradicted our project structure (e.g., suggesting MVC patterns when we were enforcing MVVM). We had to manually refactor the code to fit our clean architecture guidelines.
- **Deprecated APIs:** The LLM frequently generated code using deprecated accompanist libraries or older Material 2 APIs. We had to verify every suggestion against the official Android documentation.
- **Security Blindness:** Initial AI suggestions for the translation feature involved placing the API key directly in the client code. Recognizing this security flaw was a critical human intervention that led to the development of our backend proxy.

Ultimately, AI helped us write code faster, but it did not help us design the system. The architectural decisions, security enforcement, and user experience refinement remained a distinctly human engineering effort.